

上海电力大學

SHANGHAI UNIVERSITY OF ELECTRIC POWER

2026 年全国大学生计算机系统能力大赛

操作系统设计赛（内核实现赛道）

SWTC 初赛作品文档

面向官方评测的 RISC-V64 / LoongArch64 双架构操作系统内核

参赛学校	上海电力大学
参赛队伍	sudo_win_the_cscc
作品名称	SWTC
指导教师	徐曼
队伍成员	余子豪、王溢敏
文档日期	2026 年 6 月 30 日

初赛提交版

摘 要

SWTC 是一款面向 2026 全国大学生计算机系统能力大赛操作系统设计赛官方评测环境开发的双架构内核。项目在同一仓库维护 RISC-V64 与 LoongArch64 两条实现主线，通过统一的根构建入口生成 kernel-rv 与 kernel-la，并直接执行官方无分区 EXT4 测试盘中的预编译 ELF、动态运行时和测试脚本。

初赛阶段，SWTC 重点建设从离线源码构建、架构正确性检查、块设备发现、脚本解析、资源暂存、ELF 装载、进程执行、Linux syscall 兼容、结果输出到主动关机的完整评测闭环。RISC-V 主线以 Titanix 开源架构为基础完成比赛适配；LoongArch 主线基于 StarryX/ArceOS 组件体系完成真实架构支持。本队新增工作集中于双架构统一提交、官方评测入口、Linux ABI 兼容、故障隔离、离线依赖和验证门禁。

截至 2026 年 6 月 30 日，SWTC 当前最高且最新可见官网成绩为 1009.8903911255481。四列 basic 均为 102，BusyBox 与 Lua 四列均已计分，三列 libcbench 已计分，musl-rv libctest 为 217、LTP 为 314。该结果证明双架构构建、主要应用入口及 RISC-V functional 路径已经形成有效计分能力。

关键词：Rust；操作系统内核；RISC-V64；LoongArch64；Linux ABI；EXT4；官方评测

63	T2026104869910402	StellaOS	武汉大学	69	2026-06-30 12:00:15	102.0	102.0	102.0	102.0	54.0	54.0	53.0	53.0
64	T202610055999994	AYGNEP	南开大学	47	2026-06-23 15:23:33	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
65	T2026101269910482	一起造火箭	内蒙古大学	17	2026-06-30 11:04:58	102.0	102.0	102.0	102.0	55.0	55.0	55.0	55.0
66	T202610486999797	DWG	武汉大学	71	2026-06-30 15:01:12	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
67	T2026101269910295	再来两次	内蒙古大学	115	2026-06-30 10:55:47	102.0	102.0	102.0	102.0	53.0	54.0	53.0	54.0
68	T2026101269910207	歪比巴卜	内蒙古大学	134	2026-06-30 14:51:36	100.0	102.0	100.0	102.0	54.0	54.0	54.0	54.0
69	T2026102569910192	sudo_win_the_cscc	上海电力大学	122	2026-06-30 09:17:33	102.0	102.0	102.0	102.0	54.0	50.0	54.0	50.0
70	T2026101839910132	不想写注释队	吉林大学	99	2026-06-27 15:10:37	102.0	95.0	102.0	95.0	51.0	50.0	54.0	53.0
71	T202610487999858	httos	华中科技大学	26	2026-06-29 17:57:54	102.0	102.0	102.0	102.0	49.0	52.0	49.0	52.0
72	T2026102699910462	0xDEADBEEF	华东师范大学	182	2026-06-28 23:14:09	97.0	97.0	97.0	97.0	54.0	54.0	54.0	54.0
73	T2026100079911061	pilot.00	北京理工大学	17	2026-06-30 14:01:51	0.0	0.0	0.0	102.0	0.0	0.0	0.0	54.0
74	T2026106109910526	XCX	四川大学	33	2026-06-25 18:14:25	0.0	102.0	0.0	98.0	0.0	54.0	0.0	54.0
75	T202610701999762	NameNotFound	西安电子科技大学	26	2026-05-04 16:56:32	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0

图 0-1 sudo_win_the_cscc 官网评测列表记录

模块完成情况

模块	完成情况	验证证据
统一构建	根目录 make all 生成双架构 ELF；工具链、离线依赖与源树门禁已接入	干净导出构建通过
RISC-V64	启动、trap、进程、内存、文件系统、网络、信号及 Linux syscall 基线已接入	官方多个测试组线上计分
LoongArch64	真实 kernel-la、平台配置、用户空间、VFS、EXT4 与官方脚本入口已接入	basic/BusyBox/Lua/libcbench 线上计分
测试盘入口	识别无分区 EXT4，支持固定路径、目录遍历、脚本和 ELF 读取	glibc/musl 双目录回归
ELF 与运行时	静态/动态 ELF、PT_INTERP、argv/envp/auxv 与动态解释器暂存	basic 与 libctest 动态路径计分
应用测试	basic、BusyBox、Lua、libcbench、libctest、LTP 已形成分组执行路径	官网总分 1009.8903911255481
故障隔离	组私有工作目录、逐项 timeout、START/END 收尾与主动关机	长队列失败后仍保留前序得分
验证体系	源码完整性、vendor checksum、ELF 架构、无盘 QEMU 与官方镜像回归	提交前多层门禁
待完善项	LoongArch libctest/LTP、性能测试与更完整的跨架构行为一致性	后续按 case 扩展

完成情况采用“源码实现 - 本地回归 - 线上计分”三层证据描述。对于尚未获得线上证据的能力，文档按当前验证状态如实标注；基础开源项目能力与本队比赛适配工作在后续章节分别说明。

目 录

1. 作品概况 6

1.1 作品基本信息 6

1.2 项目人员 6

1.3 作品概述 6

2. 系统总体架构 7

2.1 双架构组织 7

2.2 核心数据流 7

2.3 代码组织与模块边界 8

2.4 双架构职责划分 8

3. 主要工作内容 9

3.1 官方构建入口与离线依赖 9

3.2 官方测试盘与脚本执行 9

3.3 Linux ABI 与进程执行 9

3.4 LoongArch64 实现与适配 9

3.5 双工具链构建系统 10

3.6 官方测试资产发现与受控执行 11

3.7 RISC-V64 内核主线 12

3.8 LoongArch64 内核主线 13

4. 完成度与验证状态 14

4.1 已建立的回归门禁 14

4.2 线上分项结果 15

4.3 分层验证矩阵 16

4.4 提交前验证原则 16

5. 项目特点与创新点 17

5.1 工程取舍 17

5.2 关键工程决策 18

5.2.1 为什么不是完全统一内核源码 18

5.3 问题定位方法与经验总结 19

6. 官网评测记录 20

7. 构建、验证与后续计划 21

7.1 构建方式 . . . 21
7.2 当前边界 . . . 21
7.3 后续计划 . . . 21
7.4 Linux ABI 覆盖结构 . . . 22
7.5 测试组推进策略 . . . 23
7.5.1 提交批次原则 . . . 23
7.6 后续里程碑与预期成果 . . . 24
7.7 阶段性结论 . . . 24

8. 参考、开源基础与致谢 . . . 25

开源与原创性说明 . . . 25

1. 作品概况

1.1 作品基本信息

项目	内容
队伍名称	sudo_win_the_cscc
操作系统名称	SWTC
支持架构	RISC-V64、LoongArch64
主要语言	Rust、Makefile、少量汇编与 Shell
官方产物	kernel-rv、kernel-la
构建方式	仓库根目录执行 make all，离线完成双架构构建
测试方向	basic、BusyBox、Lua、libcbench、libctest、LTP

1.2 项目人员

参赛身份	姓名	学校与专业
指导教师	徐曼	上海电力大学
队长、队员	余子豪	上海电力大学，计算机科学与技术（卓越计划）
队员	王溢敏	上海电力大学，计算机科学与技术（卓越计划）

1.3 作品概述

SWTC 是面向 2026
操作系统设计赛官方评测环境开发的双架构操作系统内核。项目在同一仓库、同一提交入口下维护 RISC-V64 与 LoongArch64 两条实现主线，目标是稳定生成官方要求的两个内核 ELF，并执行测试盘中的 Linux 用户态程序。

项目围绕官方无分区 EXT4 测试盘、预编译 ELF 和测试脚本构建端到端运行链路：内核启动后识别块设备，读取脚本与资源，组织测试队列，装载用户程序，提供最小 Linux ABI，按官方格式输出结果并主动关机。当前最高且最新可见官网成绩为 1009.8903911255481。

> 项目以“可构建、可启动、可执行、可计分、可复现”为初赛工程目标。

2. 系统总体架构

SWTC 采用统一提交入口与双架构实现并行的工程组织方式。根构建负责选择各自工具链、恢复离线依赖配置、检查源树完整性并生成官方产物；运行时则统一面向官方测试盘和日志协议。

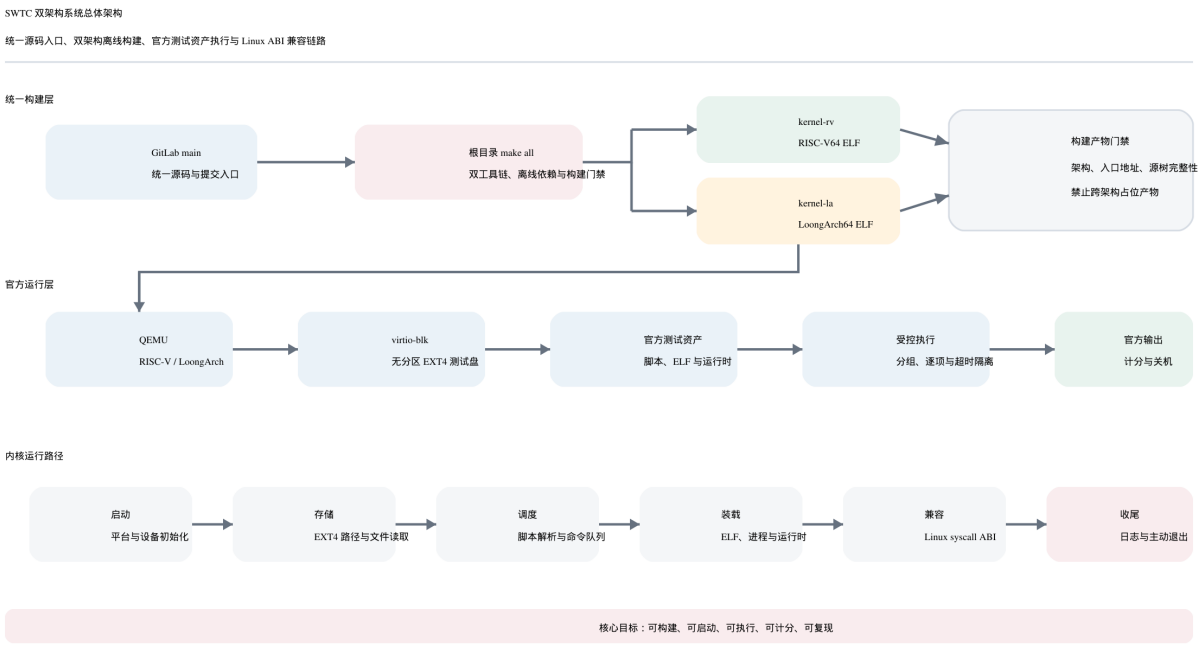


图 1 SWTC 统一构建、官方运行环境与内核运行路径

2.1 双架构组织

- RISC-V64 主线位于 SWTC/，以 Titanix 往届开源架构为基础，重点完成官方镜像入口、Linux ABI、EXT4 读取、脚本解析和测试执行适配。
- LoongArch64 主线位于 SWTC-la/，基于 StarryX/ArceOS 组件化内核进行比赛适配，重点完成工具链、真实 kernel-la、测试盘和用户态应用运行。
- 两条主线共享官方提交约束、测试组命名、日志格式、回归标准与文档记录方式。

2.2 核心数据流

阶段	输入	核心处理	输出
构建	GitLab 源码	双工具链离线构建、源树门禁	kernel-rv / kernel-la
发现	virtio-blk	识别无分区 EXT4 与固定脚本路径	测试组与资源索引
执行	脚本、ELF、运行时	staging、ELF 装载、进程与 syscall	测试结果
收尾	case 状态	官方 START/END 格式、超时隔离	计分日志与主动关机

2.3 代码组织与模块边界

SWTC 保留两套能够独立构建的内核主体，并在仓库根目录建立统一交付层。这样的组织方式允许两条主线根据架构差异选择不同内核基础，同时用相同的提交规范、测试命名和输出协议约束最终行为。

路径	职责	主要内容
Makefile / GNUmakefile	统一交付层	工具链检查、隐藏配置恢复、双架构构建、产物门禁
SWTC/kernel	RISC-V64 内核	启动、trap、进程、内存、文件系统、网络、信号、syscall 与 oscomp
SWTC/user	RISC-V 用户态	init/runtestcase、测试队列消费与官方输出协议
SWTC-la/src	LoongArch 入口	平台启动、用户应用执行、syscall 接口与 init.sh
SWTC-la/xcore	LoongArch 核心服务	VFS、fd、syscall、调度与运行时基础
SWTC-la/xmodules	组件化能力	xprocess、xsignal、xuspace、xvma、xcache、xapi
docs / tools	验证与文档	进度证据、架构图、PDF 生成与 vendor 校验脚本

2.4 双架构职责划分

- 统一部分：官方构建入口、产物名称、测试盘约定、glibc/musl 测试组、START/END 输出和主动退出。
- RISC-V 特有部分：高地址内核二进制 wrapper、oscomp 内核态测试资产 staging、runtestcase 队列执行器。
- LoongArch 特有部分：DMW/高地址映射、LoongArch 平台配置、ArceOS/StarryX 组件组合和 BusyBox init.sh 调度。
- 一致性目标：相同测试组在两条主线上使用相同资源、超时原则和结果格式，降低架构间行为偏差。

3. 主要工作内容

3.1 官方构建入口与离线依赖

- 根目录保留 Makefile 与 GNUmakefile，执行 make all 统一生成两个官方 ELF。
- RISC-V 与 LoongArch 使用各自固定 nightly 工具链，避免跨架构工具链和 target 配置相互污染。
- 针对官方提交过滤隐藏文件的约束，构建时从非隐藏目录恢复 Cargo 配置与 checksum，并对 vendor 完整性进行检查。
- LoongArch 构建默认拒绝使用 RISC-V 产物充当 kernel-la，使架构错误在编译阶段直接暴露。

3.2 官方测试盘与脚本执行

- 通过 virtio-blk 访问无分区 EXT4 测试盘，支持多级目录下普通文件与脚本的只读访问。
- 按 glibc、musl 和根目录路径定位测试组，解析脚本并形成可控的串行执行队列。
- 将 ELF、动态解释器、脚本和数据文件暂存到组私有目录，降低跨组资源污染。
- 对高风险测试采用逐 case 超时和组级 START/END 收尾，防止单项卡死导致整轮评测失效。

3.3 Linux ABI 与进程执行

- 支持 ELF PT_LOAD/PT_INTERP 处理，构造 argc、argv、envp 与 auxv，并执行动态或静态用户程序。
- 逐步实现文件描述符、路径、内存、进程、时间、信号与 futex 等 Linux syscall 兼容语义。
- 建立 fork/clone、execve、wait4 与超时控制的最小执行链，满足测试程序父子进程和线程场景。
- 根据线上日志和本地镜像结果定位缺失 ABI，按可验证的小批次扩展 LTP 覆盖。

3.4 LoongArch64 实现与适配

- 完成 LoongArch64 内核启动、离线 vendor、配置生成器与 GCC 13.2 musl 工具链适配。
- 真实 kernel-la 为 LoongArch ELF，入口地址 0x80000000；本地已验证 basic 与 BusyBox 测试组可运行并主动退出。
- 将易受官方源包过滤影响的核心依赖迁移到非隐藏目录，并在根构建中增加源树完整性门禁。
- 对测试顺序与工作目录进行隔离，避免 BusyBox 递归扫描大型 LTP 目录而中断后续测试。

3.5 双工具链构建系统

官方评测只执行仓库根目录的 make all，因此根构建既是交付入口，也是第一道质量门禁。SWTC 不让两个架构共享一个模糊的默认工具链，而是显式固定 RISC-V nightly-2025-02-01 与 LoongArch nightly-2025-05-20，并分别检查 target、rust-lld、rust-objcopy、CMake 和 loongarch64-linux-musl-gcc。

构建与交付流水线



构建项	RISC-V64	LoongArch64
Rust 工具链	nightly-2025-02-01	nightly-2025-05-20
目标三元组	riscv64gc-unknown-none-elf	loongarch64-unknown-none
最终入口	0x80200000	0x80000000
关键转换	raw binary + wrapper object + rust-lld	直接复制真实 LoongArch ELF
失败策略	缺 target/tool 立即失败	默认拒绝跨架构 placeholder

```
all: submission-info sanitize-submit-workdir build
build: build-rv build-la
build-rv: prepare-rv -> user -> kernel -> wrapper ELF
build-la: prepare-la -> strict LoongArch build -> kernel-la
```

官方会过滤隐藏文件，因此 Cargo 配置和 vendor checksum 的可恢复副本存放在非隐藏路径。构建阶段再恢复 .cargo/config.toml 与 .cargo-checksum.json；该设计不要求评测机联网，也不依赖工作区中的历史缓存。

3.6 官方测试资产发现与受控执行

RISC-V 的 oscomp 模块直接从 virtio-blk 读取 EXT4 元数据，支持 extent tree、传统块映射、目录项和多级路径解析。测试组首先按固定路径发现 glibc/ 与 musl/ 下的脚本，再读取脚本、ELF、动态解释器和数据文件，并将其安装到 tmpfs 组私有目录。

官方测试执行闭环



记录	含义	执行行为
G	进入测试组	切换组私有工作目录并输出 START
X	基础可执行文件	执行单个 ELF 并等待退出
A	argv + timeout	携带参数执行 BusyBox、Lua、bench 等
C	libctest case	按 case 输出 RUN/Pass/FAIL
L	LTP case	逐项限时并保留 case 收尾状态
E	结束标记	无论前序成败都输出组 END

执行队列采用 NUL 分隔记录，避免脚本中的空格和换行破坏边界。libctest、LTP 与 lmbench 分别设置独立 timeout；超时只终止当前项，不应吞掉整个测试组。

3.7 RISC-V64 内核主线

RISC-V64 主线位于 SWTC/kernel。其基础架构来自 Titanix GPLv3 开源项目，本队在保留许可证和来源说明的前提下，将工作重点放在 2026 官方测试资产入口、离线构建、动态运行时、结果协议及 Linux ABI 差异修复。

子系统	当前结构	比赛相关工作
启动与 trap	entry.S、trap.S、用户/内核 trap 分离	官方 QEMU 入口、主动退出与异常诊断
进程	Process/Thread、资源对象、调度与 wait	fork/clone/execve/wait4 与测试超时链路
内存	页表、VMA、COW、共享内存与用户指针检查	ELF PT_LOAD/PT_INTERP、brk/mmap 与边界语义
文件系统	VFS、fd table、FAT32/tmpfs/procfs/devfs、页缓存	EXT4 测试盘只读导入与组资源 staging
同步/时间	futex、mutex、poll、timeout 与定时任务	libcbench/LTP 所需等待和时间语义
网络/信号	TCP/UDP/Unix socket、标准信号路径	保留基础能力，初赛优先 functional 计分
oscomp	约 2200 行评测适配模块	EXT4、脚本、ELF、队列、运行时与测试组编排

RISC-V 主线的关键价值在于将已有内核能力转化为官方可观察、可计分的行为。实际适配中，很多问题并非单纯缺少一个 syscall 编号，而是 argv、结构体布局、errno、阻塞/超时、动态解释器路径和输出边界等 Linux ABI 语义细节存在偏差。

3.8 LoongArch64 内核主线

LoongArch64 主线位于 SWTC-la，以 StarryX/ArceOS 组件化体系为基础。入口程序初始化进程、挂载 VFS、建立标准输入输出，并以 /musl/busybox sh -c 执行内嵌 init.sh。xcore 提供核心服务，xprocess、xsignal、xuspace、xvma、xcache 等模块分别承载进程、信号、用户空间、虚拟内存和缓存能力。

层次	实现	比赛适配
平台	loongarch64-qemu-virt，物理入口 0x80000000	UART、GED、PCI、virtio-blk 与时钟配置
地址空间	DMW 启动窗口 + 高地址内核映射	分页开启前后保持启动代码连续执行
运行时	ArceOS axruntime/axtask/axmm/axfs-ng	真实 kernel-la 与用户态 ELF 执行
文件系统	VFS + lwext4_rust + fd/stdio	直接挂载官方测试盘并访问 glibc/musl 目录
测试编排	BusyBox init.sh	动态库链接、工作目录沙箱、逐项 timeout
离线依赖	vendor + 本地 xapi/xcore/xmodules	规避官方源包遗漏与 crates.io 网络依赖

```
fn main() {
    Process::new_init(current_task_id).build();
    vfs::init_root();
    fd::init_stdio();
    run_user_app(["/musl/busybox", "sh", "-c", init_script]);
    terminate();
}
```

LoongArch 线上已获得 basic、BusyBox、Lua 与 musl libcbench 分数；libctest/LTP 尚未形成分数，是后续最明确的增长方向。

4. 完成度与验证状态

以下状态以已完成的本地回归和可见官网结果为依据。线上得分受评测源包、排队与运行环境影响，因此文档将“实现完成”“本地验证”和“线上计分”分开描述。

测试/模块	RISC-V64	LoongArch64	当前说明
根构建	已通过	已通过	干净导出可生成两个架构正确的 ELF
basic	线上稳定得分	本地 64/64，曾线上得分	双 libc 路径已接入
BusyBox	线上得分	本地 55/55，曾线上得分	LA 已隔离工作目录
Lua	线上得分	已接入	依赖 Lua 与脚本资源 staging
libcbench	线上得分	部分接入	高风险 glibc 项已后置隔离
libctest	musl 线上得分	已接入	LA 逐 case 继续验证
LTP	稳定子集线上得分	部分接入	按 syscall 能力分批扩展
EXT4/tmpfs	已接入	已接入	只读测试盘 + 运行资源暂存
进程模型	最小 Linux ABI	组件化内核适配	尚非完整 Linux 隔离模型

4.1 已建立的回归门禁

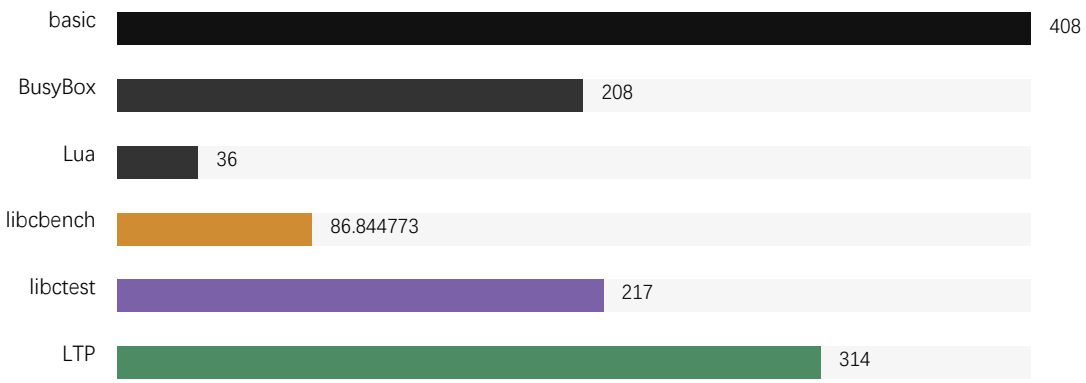
- 干净导出构建：确保源码包不依赖本机未提交文件或缓存。
- ELF 架构检查：kernel-rv 必须为 RISC-V ELF，kernel-la 必须为 LoongArch ELF。
- 无盘 QEMU 回归：没有测试盘时也能启动、输出并主动退出。
- 官方镜像回归：检查测试组 START/END、case 输出、超时行为与最终关机。
- 线上结果回读：以官网分项和串口日志更新进度，避免凭本地现象推断线上结论。

> 当前最高且最新可见官网成绩：1009.8903911255481（提交时间 2026-06-30 09:17:33，Accepted）。

4.2 线上分项结果

官网结果显示四列 basic 全部为 102，BusyBox 四列为 54、50、54、50，Lua 四列均为 9。libcbench 已在 glibc-rv、musl-la 与 musl-rv 三列计分；musl-rv 的 libctest 与 LTP 分别为 217 和 314。

官网分项原始计分



注：图中为官网各测试组原始分项；平台总分按官方计分规则显示为 1009.8903911255481。

测试组	glibc-la	glibc-rv	musl-la	musl-rv	分项
basic	102	102	102	102	408
BusyBox	54	50	54	50	208
Lua	9	9	9	9	36
libcbench	0	30.3442	29.3344	27.1661	86.8448
libctest	-	-	0	217	217
LTP	0	0	0	314	314

表内数值按官网得分详情原样记录；官网最终总分显示为 1009.8903911255481。

4.3 分层验证矩阵

排队时间可能达到数小时，因此每次提交前需要尽量在本地消除可以确定的问题。SWTC 将验证分为源码、构建、产物、启动、功能、协议和线上七层；后一层不替代前一层。

层级	检查内容	失败时含义	提交门禁
源码	关键目录、Cargo.toml、vendor checksum	官方导出后可能缺文件	必须通过
构建	离线 make all，不访问网络	工具链或隐藏配置依赖	必须通过
产物	ELF 架构与入口地址	kernel-la placeholder 或链接错误	必须通过
启动	官方 QEMU 命令、无盘主动退出	启动栈、地址、设备或关机错误	必须通过
功能	官方 EXT4 镜像与分组 case	脚本、ELF、syscall 或运行时语义错误	目标组通过
协议	START/END、RUN、Pass/FAIL 与 !TEST FINISH!	程序运行但计分器无法识别	必须通过
线上	总分、四列分项与串口日志	环境差异或未覆盖 ABI	形成新证据

4.4 提交前验证原则

- 先验证无盘启动和主动退出，避免测试盘逻辑掩盖基础启动回归。
- 再验证 basic、BusyBox、Lua 等稳定组，确认保分路径未被新功能破坏。
- 新 LTP case 只有在 syscall 行为和错误路径明确时才加入 allowlist。
- 高风险性能项后置运行，不能截断前面已经可计分的 functional 组。
- 线上结果与对应提交建立关联，避免不同提交批次的页面结果混淆。

5. 项目特点与创新点

方向	实现与价值
双架构统一提交	在同一仓库维护 RISC-V64 与 LoongArch64 两套内核、依赖和工具链，根构建统一产出官方要求的两个 ELF。
测试脚本受控执行	不依赖完整发行版环境，将官方脚本解析为内核可调度的命令队列，统一管理资源、顺序、超时和日志。
评测约束工程化	围绕隐藏文件过滤、离线构建、源树缺失和长排队等现实约束，设计非隐藏依赖、完整性门禁和干净导出验证。
日志驱动 ABI 演进	根据真实 ELF 行为和线上日志逐项补齐 Linux syscall 语义，以可回归的小批次扩展 functional 测试覆盖。
真实架构产物防线	禁止用 RISC-V ELF 冒充 kernel-la，构建阶段检查源树与 ELF 架构，使错误尽早暴露。
测试组故障隔离	通过组私有工作目录、逐 case 超时和高风险项后置，减少单个测试对整轮成绩的级联影响。

5.1 工程取舍

初赛阶段优先保证官方入口、Linux ABI 和 functional 测试链路，不将网络、图形界面等展示性功能置于计分闭环之前。对于尚未稳定的 iozone、lmbench 和大规模 LTP，采用后置、分批和可回滚策略，以维护已验证基线的稳定性。

> 创新重点不在堆叠功能数量，而在双架构、真实测试资产和受限评测环境下建立稳定、可解释、可复现的内核运行链路。

5.2 关键工程决策

决策	问题背景	采用方案	结果
统一根入口	评测固定执行 make all	根 Makefile/GNUmakefile 编排双架构	交付行为单一明确
离线依赖	评测环境不能访问 crates.io	vendor + 非隐藏 checksum/config 备份	干净导出可离线构建
严格 kernel-la	占位产物会在运行期静默丢分	默认构建失败而非复制 kernel-rv	架构错误提前暴露
固定路径优先	大镜像递归扫描成本高	优先 glibc/musl 固定脚本路径	缩短启动与发现时间
队列执行器	RISC-V 不依赖完整 shell	内核 staging + 用户态记录消费	参数、超时和边界可控
组级沙箱	BusyBox find/du 会递归大目录	复制必要资源到 /tmp 私有目录	避免扫描 LTP 树导致 trap
高风险后置	性能测试可能异常退出或超时	functional 优先，bench 后置限时	保留已经获得的分数

5.2.1 为什么不是完全统一内核源码

RISC-V 与 LoongArch 主线的开源基础不同，短期强行合并会引入大范围回归，且无法直接提高初赛得分。项目选择统一交付接口、测试协议和验证标准，而允许底层实现保持架构适配所需差异。这是一种面向截止时间与风险的工程取舍。

> 统一的对象是可观察行为与提交质量，而不是为了目录形式一致去重写已经能工作的内核。

5.3 问题定位方法与经验总结

内核评测问题往往跨越构建环境、QEMU、文件系统、动态链接器、用户程序和计分器。SWTC 将问题定位拆解为连续证据链，每一步只验证一个假设，避免仅根据最终分数推断底层原因。

现象	优先核查证据	判断方向
Compile Error	根 Makefile 标记、缺失路径、Cargo checksum	入口/源包/离线依赖问题，不是运行时 ABI
QEMU 无输出	ELF 架构、入口地址、串口和启动栈	产物或平台初始化错误
找到脚本但 0 分	START/END、真实 case 输出、计分格式	协议或执行器未真正运行 ELF
execve 返回 ENOENT	PT_INTERP 与动态库实际路径	动态解释器缺失，而非主 ELF 不存在
某组后全部归零	最后一条串口日志与 trap 类型	当前组截断后续，应先隔离/后置
本地通过线上失败	干净导出、隐藏文件过滤、工具链版本	本机缓存或未提交文件依赖

- 建立最小复现：先用单个 ELF、单个脚本和最小测试盘确认入口。
- 同时记录正向证据和负向证据：不仅记录通过项，也记录暂缓打开某组测试的工程依据。
- 分数回退时优先比较测试组执行顺序与最后有效日志，不以总分单点判断根因。
- 将平台异常与代码问题分开：源包工作目录、重复压缩文件等平台现象需要独立记录。
- 文档与代码同步：新增 syscall、构建门禁和测试组都应留下验证方式与已知限制。

> 该定位方法的目标，是让每次线上提交都建立在明确证据与可回归验证之上。

6. 官网评测记录

下图为 2026 年 6 月 30 日官网评测列表截图。红框行显示参赛队伍 sudo_win_the_cscc，学校为上海电力大学；多个分项列已获得稳定结果，证明 SWTC 已能进入官方评测链路并执行双架构相关测试。

63	T2026104869910402	StellaOS	武汉大学	69	2026-06-30 12:00:15	102.0	102.0	102.0	102.0	54.0	54.0	53.0	53.0
64	T2026100559999994	AYGNEP	南开大学	47	2026-06-23 15:23:33	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
65	T2026101269910482	一起造火箭	内蒙古大学	17	2026-06-30 11:04:58	102.0	102.0	102.0	102.0	55.0	55.0	55.0	55.0
66	T2026104869997797	DWG	武汉大学	71	2026-06-30 15:01:12	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
67	T2026101269910295	再来两次	内蒙古大学	115	2026-06-30 10:55:47	102.0	102.0	102.0	102.0	53.0	54.0	53.0	54.0
68	T2026101269910207	歪比巴卜	内蒙古大学	134	2026-06-30 14:51:36	100.0	102.0	100.0	102.0	54.0	54.0	54.0	54.0
69	T2026102569910192	sudo_win_the_cscc	上海电力大学	122	2026-06-30 09:17:33	102.0	102.0	102.0	102.0	54.0	50.0	54.0	50.0
70	T2026101839910132	不想写注释队	吉林大学	99	2026-06-27 15:10:37	102.0	95.0	102.0	95.0	51.0	50.0	54.0	53.0
71	T2026104879998858	httos	华中科技大学	26	2026-06-29 17:57:54	102.0	102.0	102.0	102.0	49.0	52.0	49.0	52.0
72	T2026102699910462	0xDEADBEEF	华东师范大学	182	2026-06-28 23:14:09	97.0	97.0	97.0	97.0	54.0	54.0	54.0	54.0
73	T2026100079911061	pilot.00	北京理工大学	17	2026-06-30 14:01:51	0.0	0.0	0.0	102.0	0.0	0.0	0.0	54.0
74	T2026106109910526	XCX	四川大学	33	2026-06-25 18:14:25	0.0	102.0	0.0	98.0	0.0	54.0	0.0	54.0
75	T202610701999762	NameNotFound	西安电子科技大学	26	2026-05-04 16:56:32	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0

图 2 sudo_win_the_cscc 官网评测列表记录（2026-06-30）

评测成绩是阶段性工程证据，不等同于完整 Linux 兼容度。项目后续仍将通过本地官方镜像回归和线上分项日志交叉验证各项能力。

7. 构建、验证与后续计划

7.1 构建方式

```
cd /mnt/d/os-kernel-contest
make all
```

- kernel-rv：RISC-V executable ELF，入口地址 0x80200000。
- kernel-la：LoongArch executable ELF，入口地址 0x80000000。
- 构建默认使用仓库内离线依赖，不要求评测机访问 crates.io。

7.2 当前边界

边界	后续处理
进程与资源隔离仍是最小实现	继续完善 per-process fd、父子资源语义和独立内核栈
LoongArch libctest/LTP 覆盖不足	按 case 日志补齐 syscall 与运行时语义
高风险性能测试可能截断后续组	保持后置、限时和独立工作目录
双架构实现基础不同	统一测试协议、构建门禁和文档，逐步收敛公共行为

7.3 后续计划

优先级	工作项	验收标准
P0	稳定双架构官方构建与启动	两个 ELF 架构正确，QEMU 可进入内核并主动退出
P1	巩固已验证高分路径	basic/BusyBox/Lua/libcbench 分项保持稳定
P2	扩展 LA libctest 与 LTP	逐 case 可解释，新增 syscall 有本地和线上证据
P3	完善进程与文件描述符模型	fork/exec/wait 与多程序资源隔离回归通过

7.4 Linux ABI 覆盖结构

RISC-V syscall 入口按文件、内存、进程、资源、信号、同步、时间、网络和设备拆分。初赛适配不仅增加 syscall 编号，也处理结构体布局、flag、errno、短读写、阻塞与超时等语义。下表列出当前对官方测试最关键的能力类别。

类别	代表能力	关联测试	注意事项
文件与路径	openat/read/write/stat/getdents/rename/truncate/link	basic、BusyBox、LTP	dirfd、flag、软链接与错误码
向量与复制	readv/writev/preadv/pwritev/sendfile	LTP、libcbench	IOV_MAX、短读写与 offset 更新
内存	brk/mmap/munmap/mprotect/shm	basic、libctest	页对齐、权限、COW 与用户指针
进程	clone/fork/execve/wait4/exit	basic、lmbench、LTP	父子资源、退出状态与动态加载
时间	clock_gettime/nanosleep/timerfd/gettimeofday	libcbench、LTP	tv_nsec 校验与超时中断
同步与复用	futex/poll/select/pselect/eventfd/flock	libcbench、LTP	等待队列、nonblock 与 wake 语义
信号	sigaction/sigprocmask/kill/sigreturn	libctest、LTP	mask、handler 与 trap 上下文
网络	socket/bind/connect/send/recv/poll	后续网络与应用	初赛 functional 优先级较低

> syscall 实现存在不代表所有边界 case 已兼容。LTP allowlist 只选择已经理解其前置条件、失败影响和验证方法的 case。

7.5 测试组推进策略

测试组	价值	当前策略	下一验收点
basic	启动、基础 syscall 与双 libc 基线	四列 102，作为每次回归门禁	保持满分且不增加耗时
BusyBox	文件系统与常用命令组合	私有工作目录，避免递归大树	四列稳定不回退
Lua	解释器、文件与内存综合场景	完整资源 staging 后运行脚本	四列 9 保持
libcbench	malloc、线程、futex 与性能路径	保留三列已计分，高风险项后置	补齐 glibc-la 且不截断
libctest	libc 语义密集、单项价值高	逐 case、静态/动态分开、独立 timeout	LoongArch 首批可识别 case
LTP	大规模 Linux ABI 验证	allowlist 分批扩展，失败继续	LoongArch 非零并提高 RV 稳定项
lmbench/iozone	性能与复杂文件路径	全部 functional 后置，严格限时	不影响前序分数后再扩大
网络/实时	应用与实时能力扩展	初赛末期不作为主攻方向	核心 ABI 稳定后再进入

7.5.1 提交批次原则

- 一次提交必须有清晰的目标测试组和可解释的预期增量。
- 涉及构建、文件系统、执行顺序和 ABI 的改动分开提交，便于回退与归因。
- 新增大批 case 前先确认 runner 格式与一个代表 case，避免整组无效。
- 任何可能异常退出或长时间阻塞的测试必须有独立 timeout 和 END 收尾。

7.6 后续里程碑与预期成果

阶段	重点工作	预期成果
M1 保分	冻结 1009.8903911255481 对应稳定执行顺序；保持双架构构建门禁	每次提交 basic/BusyBox/Lua 不回退
M2 LA functional	从首个 LoongArch libctest/LTP 日志补齐 syscall 与运行时	LA libctest/LTP 由 0 变为可持续增长
M3 资源语义	收敛 fd、fork/exec/wait、signal 与超时的跨架构差异	多进程和动态 libc case 更稳定
M4 性能路径	在 functional 完成后恢复 lmbench/iozone 小子集	性能组不再截断前序测试
M5 文档归档	整理来源、架构、验证矩阵和已知限制	提交材料与代码证据一致

7.7 阶段性结论

SWTC 已完成从“能够启动的两套内核”到“能够在同一官方提交中构建、发现测试资产、执行真实 Linux 用户程序并获得四列分项”的关键跨越。1009.8903911255481
分不是终点，而是一条具有工程证据的高分基线。后续工作的核心是提高 LoongArch functional 覆盖，并在不牺牲已有稳定性的前提下逐步扩大 Linux ABI 兼容范围。

> 项目现阶段最重要的资产，是已经建立的双架构交付链、可重复验证流程以及能够从真实日志解释得分变化的工程方法。

8. 参考、开源基础与致谢

SWTC 在遵守原项目许可证的前提下学习并复用开源操作系统的架构基础。RISC-V 主线以 Titanix 往届 GPLv3 开源项目为基础；LoongArch 主线参考 StarryX 与 ArceOS 组件化设计。本队增量主要集中在 2026 官方评测适配、双架构构建、Linux ABI 修复、测试调度和离线构建工程化。

[1] Titanix：https://github.com/greenhandzpx/Titanix

[2] StarryX：https://github.com/Anekoique/StarryX

[3] NoAxiom-OS：https://github.com/NoAxiom/NoAxiom-OS

[4] NPUcore-IMPACT：https://github.com/Fediory/NPUcore-IMPACT

[5] UMI：https://github.com/js2xxx/umi

[6] Alien：https://github.com/Godones/Alien

[7] NPUcore：https://github.com/echodjx/NPUcore

[8] UltraOS：https://github.com/xiyurain/UltraOS

[9] Del0n1x / myOS：https://github.com/Ya0rk/myOS

[10] Chronix：https://github.com/PACTHEMAN123/Chronix

开源与原创性说明

项目保留所使用开源组件的许可证和来源说明，不直接复制未授权实现。文档所述比赛适配、测试组织、构建门禁和 ABI 修复以本仓库提交记录与验证材料为依据。

- 文档结束 -